

Le langage procédural en SQL sous PostGres

PL/pgSQL

(Procedural Language / postgresQL Structured Query Language)

Correction Détaillée & Guide Pédagogique - TP 1 (Les Fonctions)

BUT 2 Informatique - IUT d'Arles (Aix-Marseille Université)

Introduction au PL/pgSQL & Structure d'un bloc

■ Pourquoi PL/pgSQL ?

- Le SQL standard est *non procédural* (déclaratif) : impossible de gérer des boucles ou des variables locales.
- PL/pgSQL est une extension côté serveur très proche du PL/SQL d'Oracle, permettant d'exécuter des algorithmes complets directement au cœur du SGBD.

■ Structure obligatoire en bloc :

```
CREATE OR REPLACE FUNCTION nom_fonction(arguments) RETURNS type_retour AS $$  
DECLARE  
    -- Déclarations obligatoires des variables locales avant BEGIN  
BEGIN  
    -- Séquence d'instructions algorithmiques et requêtes SQL  
    RETURN expression;  
END; $$ LANGUAGE plpgsql;
```

■ Rôle fondamental des délimiteurs \$\$ (Vidéo Erreur 1) :

- Sans les \$\$, le moteur SQL analyserait chaque point-virgule ; à l'intérieur du bloc comme la fin prématurée de la commande CREATE FUNCTION et lèverait une erreur de syntaxe.

Variables et Règles d'Affectation en PL/pgSQL

■ Typage des variables (DECLARE) :

- **Scalars** : integer, numeric, varchar(n), boolean...
- **%TYPE** : copie automatique du type d'une colonne en base.
v_sal employes.salaire%TYPE;
- **%ROWTYPE** : variable ligne structurée sur une table entière.
emp employes%ROWTYPE; → accessible via emp.nom
- **RECORD** : variable ligne générique sans structure fixe prédéfinie.
Sa sous-structure s'adapte à l'assignation.

■ L'affectation : règles d'or (Diapo 25 à 29) :

- Affectation scalaire : variable := expression;
- Depuis une requête : **SELECT ... INTO variable FROM ...;**

ATTENTION (Vidéo Erreur 3 du cours) :

Dans un bloc PL/pgSQL, un SELECT ne peut jamais être orphelin ! Il doit **toujours** stocker son résultat dans une variable avec INTO, sous peine d'erreur de compilation/exécution.

Les 4 Modes de Retour d'une Fonction (Diapo 43 à 52)

- Il convient de bien distinguer le nombre de lignes et de colonnes retournées :

Type de résultat	Déclaration RETURNS	Instruction dans la fonction	Syntaxe d'appel SQL
1 valeur scalaire (1 ligne, 1 colonne)	RETURNS type (ex: numeric, varchar)	RETURN expression;	SELECT nomfonct(...);
1 ligne composite (1 ligne, N colonnes)	RETURNS record (ou %ROWTYPE)	RETURN rec;	SELECT * FROM nomfonct(...) AS (col1 type, col2 type);
Table scalaire (N lignes, 1 colonne)	RETURNS SETOF type	Dans une boucle : RETURN NEXT valeur;	SELECT * FROM nomfonct(...);
Table composite (N lignes, N colonnes)	RETURNS SETOF record	Dans une boucle : RETURN NEXT rec;	SELECT * FROM nomfonct(...) AS (col1 type, col2 type);

Consigne Début de TP (Rappel Oral du Professeur sur la Syntaxe d'Appel) :

- **Si ça retourne plus d'une colonne (RECORD)** : obligation absolue de spécifier les types :

```
SELECT * FROM info_emp(12) AS (col1 type, col2 type...);
```
- **Et si c'est plus d'une ligne (mais 1 seule colonne scalaire)** : l'appel direct sans AS est parfait :

```
SELECT * FROM info_emp(12);
```

Exercice 1 : Calcul du Salaire Moyen (Questions A & B)

- A. Écrire une fonction moySalaire sans paramètre renvoyant le salaire moyen :

```
CREATE OR REPLACE FUNCTION moySalaire() RETURNS NUMERIC AS $$  
DECLARE  
    v_moyenne NUMERIC; -- Variable locale réceptrice  
BEGIN  
    -- Calcul de la moyenne arithmétique et affectation via INTO  
    SELECT AVG(salaire) INTO v_moyenne FROM employes;  
  
    RETURN v_moyenne; -- Renvoi du résultat scalaire unique  
END; $$  
LANGUAGE plpgsql;
```

- B. Tester cette fonction en l'appelant directement :

```
SELECT moySalaire() AS salaire_moyen;
```

- Résultat obtenu : 22700 (valeur exacte attendue par le sujet).

Décorticage BUT2 : La fonction ne prend aucun argument () et renvoie un scalaire unique avec RETURNS NUMERIC. On utilise la fonction d'agrégation standard AVG(salaire).

Exercice 1 : Utilisation de la Fonction en Requêtes (C & D)

■ C. Employés qui gagnent plus que le salaire moyen :

```
SELECT nom, prenom, salaire
FROM employes
WHERE salaire > moySalaire()
ORDER BY salaire DESC;
```

→ Remplace avantageusement une sous-requête lourde WHERE `salaire > (SELECT AVG(salaire) ...)`.

■ D. Employés dont le salaire est égal au salaire moyen à 10% près (avec ABS) :

```
SELECT nom, prenom, salaire
FROM employes
WHERE ABS(salaire - moySalaire()) <= 0.10 * moySalaire()
ORDER BY salaire;
```

Comprendre simplement avec les chiffres concrets du TP :

- Salaire moyen calculé : **22 700 €**
 - Marge de 10% tolérée : **2 270 €** (soit $0.10 * 22700$)
 - Salaires acceptés : entre **20 430 €** (90%) et **24 970 €** (110%)
 - **Pourquoi la fonction ABS() ?** Pour calculer l'écart sans se soucier du signe !
- Exemple :* Pour un employé à 21 000 €, l'écart brut est $21\,000 - 22\,700 = -1\,700$ €.
Grâce à `ABS(-1700)`, on obtient **1 700 €**, ce qui est bien $\leq 2\,270$ € !

Exercice 2 : Recherche du Nom de Département

- **Objectif** : Fonction recevant un numéro d'employé (noemp) et renvoyant le nom de son département :

```
CREATE OR REPLACE FUNCTION departement(p_noemp NUMERIC) RETURNS VARCHAR AS $$  
DECLARE  
    -- Utilisation de %TYPE pour hériter automatiquement du type de la colonne  
    v_nom_dept departement.nom%TYPE;  
BEGIN  
    SELECT d.nom INTO v_nom_dept  
    FROM departement d  
    JOIN employes e ON e.nodept = d.nodept  
    WHERE e.noemp = p_noemp;  
  
    RETURN v_nom_dept;  
END; $$  
LANGUAGE plpgsql;
```

- **Tests de la fonction :**

```
SELECT departement(1); -- Administration (nodept 50)  
SELECT departement(2); -- Distribution (nodept 41)  
SELECT departement(4); -- Finance (nodept 10)
```

Exercice 3 : Collègues - Analyse du Piège Régional

■ Mise en garde expresse du sujet :

« Attention il peut y avoir le même département dans des régions différentes. L'employé ne devra pas faire partie de la liste des collègues. »

■ Analyse des données réelles de la table `departement` :

- ☐ `nodept = 41` → 'Distribution' (Région 1)
- ☐ `nodept = 42` → 'Distribution' (Région 2)
- ☐ `nodept = 43` → 'Distribution' (Région 3), etc.

Erreur classique à éviter :

Si vous faites une jointure sur le nom du département (`WHERE nom = 'Distribution'`), vous allez regrouper des employés qui travaillent dans des régions géographiques complètement distinctes !

Règle : Les vrais collègues sont ceux qui partagent le **même numéro de département (nodept)**.

Spécifications techniques imposées par le sujet :

- Utiliser une variable de type `RECORD` (structure ligne flexible).
- Renvoyer chaque résultat ligne par ligne avec `RETURN NEXT nomrecord`.

Exercice 3 : Code & Syntaxe d'Appel de collègues()

```
CREATE OR REPLACE FUNCTION colleagues(p_noemp NUMERIC) RETURNS SETOF RECORD AS $$
DECLARE
    v_nodept employees.nodept%TYPE;
    rec_college RECORD;
BEGIN
    -- 1. Récupération du département de l'employé
    SELECT nodept INTO v_nodept FROM employees WHERE noemp = p_noemp;

    -- 2. Parcours des collègues (hors lui-même)
    FOR rec_college IN
        SELECT nom, prenom FROM employees
        WHERE nodept = v_nodept AND noemp <> p_noemp ORDER BY nom
    LOOP
        RETURN NEXT rec_college;
    END LOOP;
    RETURN;
END; $$ LANGUAGE plpgsql;
```

- Syntaxe d'appel obligatoire (Vidéo Erreur 5 du cours) :

```
SELECT * FROM colleagues(16) AS (nom VARCHAR, prenom VARCHAR);
```

- Résultat pour Samira Traibien (noemp 16, dept 41) :

Fonfec Sophie, Zeublouze Agathe, Zoudankou Debbie (*Samira est bien exclue !*).

Exercice 4 : Supérieurs - L'Association Réflexive

■ Comprendre le modèle de données :

- La table employes possède une clé étrangère réflexive : nosupr → employes(noemp).
- Un employé pointe vers son supérieur qui est lui-même un employé. Le sommet de l'arbre est le Président, dont nosupr IS NULL.

■ Étape 1 (Conseil du sujet) : Fonction du supérieur direct :

```
CREATE OR REPLACE FUNCTION supérieur_direct(p_noemp NUMERIC) RETURNS RECORD AS $$  
DECLARE  
    rec_supr RECORD;  
BEGIN  
    SELECT supr.nom, supr.prenom INTO rec_supr  
    FROM employes e JOIN employes supr ON e.nosupr = supr.noemp  
    WHERE e.noemp = p_noemp;  
  
    RETURN rec_supr;  
END; $$ LANGUAGE plpgsql;
```

Exemple de chaîne pour Samira Traibien (noemp 16) :

Samira (16) → Chef direct : Zoudanlkou (6) → Chef : Zeublouze (2) → Chef : Patamob (1) → NULL.

Exercice 4 : Remontée Complète avec Boucle WHILE

```
CREATE OR REPLACE FUNCTION superieurs(p_noemp NUMERIC) RETURNS SETOF RECORD AS $$
DECLARE
    v_current_supr employes.nosupr%TYPE;
    rec_supr RECORD;
BEGIN
    -- Initialisation avec le supérieur direct
    SELECT nosupr INTO v_current_supr FROM employes WHERE noemp = p_noemp;

    -- Boucle WHILE tant qu'on n'a pas atteint le Président (nosupr IS NULL)
    WHILE v_current_supr IS NOT NULL LOOP
        SELECT nom, prenom INTO rec_supr FROM employes WHERE noemp = v_current_supr;
        RETURN NEXT rec_supr; -- Émission du supérieur

        -- Remontée d'un cran : on prend le chef du chef
        SELECT nosupr INTO v_current_supr FROM employes WHERE noemp = v_current_supr;
    END LOOP;
    RETURN;
END; $$ LANGUAGE plpgsql;
```

Test d'appel : `SELECT * FROM superieurs(16) AS (nom VARCHAR, prenom VARCHAR);`
→ Retourne dans l'ordre : Zoudankou Debbie, Zeublouze Agathe, Patamob Adhémar.

Synthèse : Les 5 Pièges Signalés dans le Cours

- **1. Les délimiteurs \$\$ (Vidéo 1) :**
Sans les dollars, le parseur SQL prend le premier point-virgule ; pour la fin de la fonction.
- **2. PAS DE POLYMORPHISME (Consigne impérative du début de TP & Vidéo 2) :**
À ÉVITER CATÉGORIQUEMENT : En PL/pgSQL, si vous changez les paramètres d'une fonction, CREATE OR REPLACE n'écrase PAS l'ancienne : il crée une **surcharge** (polymorphisme). On se retrouve avec deux fonctions de même nom avec des paramètres différents, donc qui ne font pas la même chose !
Règle d'or : Ne jamais surcharger. Toujours exécuter `DROP FUNCTION nom(types);` avant de modifier une fonction.
- **3. SELECT orphelin sans variable INTO (Vidéo 3) :**
Un SELECT qui n'affecte pas son résultat via INTO lève une erreur immédiate en PL/pgSQL.
- **4. SQL Dynamique sans EXECUTE (Vidéo 4) :**
Si un nom de table ou de colonne est une variable, le SQL statique échoue. Il faut utiliser EXECUTE '...' || quote_ident(...).
- **5. Appel d'une fonction RECORD (Vidéo 5) :**
Une fonction renvoyant un RECORD exige impérativement la clause AS (col1 type1, col2 type2) lors du SELECT * FROM fonct().

Checklist pour Réussir ses Fonctions PL/pgSQL

Les réflexes incontournables de l'étudiant en BUT 2 :

- **Zéro polymorphisme** : Pas de fonctions homonymes à paramètres différents. Toujours faire un DROP FUNCTION en cas de refactorisation.
- **Syntaxe d'appel maîtrisée** : AS (col1 type, ...) uniquement si plusieurs colonnes (RECORD). Si une seule colonne sur plusieurs lignes, SELECT * FROM f(...) suffit.
- Privilégier systématiquement **table.colonne%TYPE** aux types codés en dur.
- Toujours vérifier qu'un **SELECT** est suivi de **INTO**.
- Bien choisir le mode de retour :
 - ☐ Une seule valeur → RETURNS type + RETURN val;
 - ☐ Plusieurs lignes → RETURNS SETOF type/record + RETURN NEXT ligne;
- Dans une boucle WHILE, s'assurer que la condition évolue bien à chaque itération (pas de boucle infinie).
- Toujours tester ses fonctions avec des cas nominaux et des cas limites (ex: le Président sans supérieur).

Fin de la présentation — Bon travail sur vos TP de PL/pgSQL !